

Lab 3: Architectural Pattern Selection - Written Justification

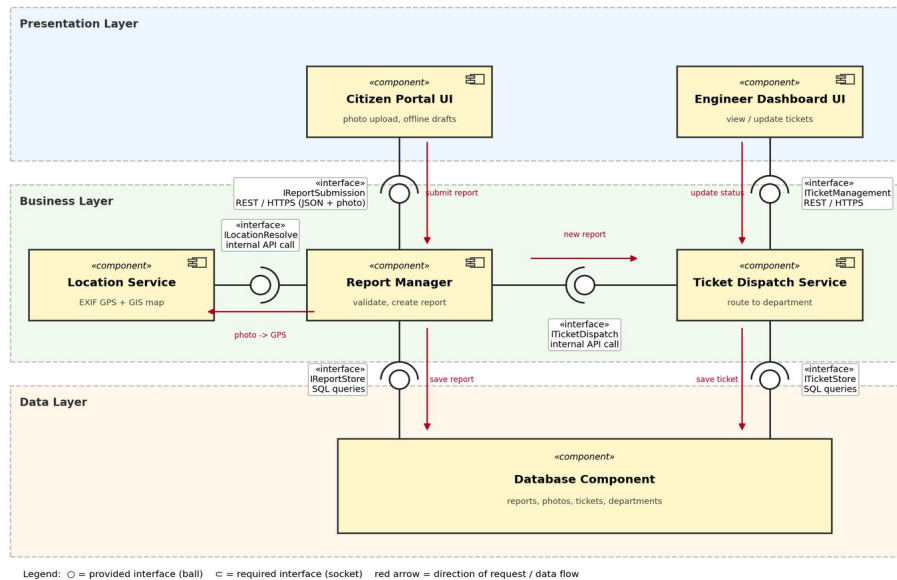
Problem Statement #29: Municipal Infrastructure Damage Reporting App

Architecture Selection: We chose **Layered Architecture** for the Municipal Infrastructure Damage Reporting App.

Architectural Choice

The system is split into three layers: a Presentation Layer (Citizen Portal UI, Engineer Dashboard UI), a Business Layer (Report Manager, Location Service, Ticket Dispatch Service) and a Data Layer (Database Component). Each layer only talks to the layer directly below it through defined interfaces.

Municipal Infrastructure Damage Reporting App - Layered Architecture (UML Component Diagram)



Two Reasons for This Choice

- 1. **Clear split of responsibilities.** The app has three natural jobs: showing screens to two kinds of users (citizens and engineers), applying the reporting rules (read GPS from the photo, create the report, dispatch a ticket to the Road, Streetlight or Water department), and storing data. Layered architecture maps directly onto these jobs, and both user screens can reuse the same rules in the Business Layer.
- 2. **Fits a simple, step-by-step workflow.** A report always moves in one direction: submit, locate, create ticket, engineer resolves. Layered calls follow this order naturally. Microservices would add network calls, deployment and data-consistency problems that this project size does not need, and Client-Server alone gives no structure for the dispatch logic.

Security Advantage

Only the Business Layer can reach the database. The two UIs never send queries directly, so every request must pass validation and access checks first (for example, only a Municipal Engineer can update a ticket, and uploaded photos are checked for type and size). This limits the damage from SQL injection or a bypassed screen, and all traffic between UI and server uses HTTPS.

Performance Benefit

Because the layers run together in one deployment, calls between the Report Manager, Location Service and Ticket Dispatch Service are direct calls with no extra network delay. The Business Layer can also cache the department routing table and map lookups, and the Citizen Portal keeps offline drafts on the phone and syncs them when the network returns, so users are never blocked waiting on the server.

Trade-off: layers are harder to scale one by one than microservices. This is acceptable because the load is mostly report submissions, and the whole app can be scaled by running more copies behind a load balancer.